Algorithmes et Pensée Computationnelle

Algorithmes de tri et Complexité - exercices basiques

Le but de cette série d'exercices est d'aborder les notions présentées durant la séance de cours. Cette série d'exercices sera orientée autour des points suivants :

1. La récursivité
2. La complexité des algorithmes
3. Les algorithmes de tri

Les exercices de cette séance seront réalisés en Java et Python.

Le temps indiqué (🕒) est à titre indicatif.

1 Récursivité

Le but principal de la récursivité est de résoudre un gros problème en le divisant en plusieurs petites parties à résoudre.

Question 1: (🕒 *10 minutes*) **Somme des chiffres**
Écrivez une fonction récursive en Python et en Java qui prend un entier en paramètre et retourne la somme des chiffres dont il est composé. Par exemple, la somme des chiffres de 126 est : 1+2+6 = 9.

Conseil

Pour obtenir le chiffre des unités d'un nombre, utilisez l'opérateur modulo `%`. Par exemple : $42 \bmod 10 = 2$, car le reste de la division entière de 42 par 10 est 2. Ensuite, pour obtenir le chiffre 4, effectuez une division entière par 10 : en Python, `42 // 10` vaut 4, en Java, `42 / 10` vaut 4, car les deux opérandes sont des entiers.

Solution

Python :

```python
def sum_digits(number):
    if number == 0:
        return 0
    else:
        return (number % 10) + sum_digits(number // 10)

print(sum_digits(126))
```

Java :

```java
public class SumDigits {
    public static int sum_digits(int number) {
        if (number == 0) {
            return 0;
        } else {
            return (number % 10) + sum_digits(number / 10);
        }
    }

    public static void main(String[] args) {
        System.out.println(sum_digits(126));
    }
}
```

2 Complexité

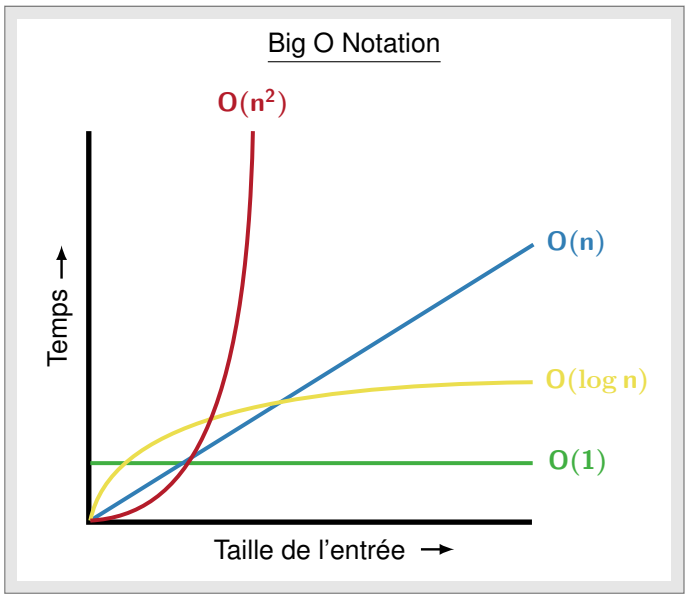


FIGURE 1 – Représentation de complexités temporelles

Indiquez en une phrase, ce que font ces algorithmes ci-dessous et calculez leur complexité temporelle avec la notation O. Le code est écrit en Python et en Java.

Question 2: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?
Python :

```python
# Entrée: n un nombre entier
def algo(n):
    s = 0
    for i in range(10*n):
        s += i
    return s
```

Java :

```java
public static int algo(int n) {
    int s = 0;
    for (int i=0; i < 10*n; i++){
        s += i;
    }
    return s;
}
```

(A) $O(n)$
(B) $O(n^3)$
(C) $O(\log(n))$
(D) $O(n^n)$

Conseil

Rappelez-vous que la notation $O()$ sert à exprimer la complexité d'algorithmes dans le **pire des cas**. Les règles suivantes vous seront utiles. Pour n étant la taille de vos données, on a que :

1. Les constantes sont ignorées : $O(2n) = 2 * O(n) = O(n)$
2. Les termes dominés sont ignorés : $O(2n^2 + 5n + 50) = O(n^2)$

Solution

L'algorithme est composé d'une boucle qui incrémente une variable `s`. Il effectue l'opération d'addition $10 * n$ fois et par conséquent a une complexité de $O(n)$.

Question 3: (*10 minutes*) **Complexité**
Quelle est la complexité des programmes ci-dessous (n étant la longueur de la liste `L`) ?
Python :

```python
# Entrée: L est une liste de nombres entiers et M un nombre entier
def algo(L, M):
    i = 0
    while i < len(L) and L[i] <= M:
        i += 1
    s = i - 1
    return s

```

Java :

```java
public static int algo(int[] L, int M) {
    int i = 0;
    while (i < L.length && L[i] <= M){
        i += 1;
    }
    int s = i - 1;
    return s;
}

```

(A) $O(n^3)$
(B) $O(\log(n))$
(C) $O(n)$
(D) $O(n^n)$

Solution

L'algorithme est composé d'une boucle `while` qui va parcourir une liste `L` jusqu'à trouver une valeur dans la liste qui est supérieure à `M`. Ainsi, dans le pire des cas, l'algorithme parcourt toute la liste, et a donc une complexité de $O(n)$, n étant la taille de la liste.

Question 4: (*10 minutes*) **Complexité**
Quelle est la complexité des programmes ci-dessous ?
Python :

```python
#Entrée: L et M sont 2 listes de nombres entiers
def algo(L, M):
    n = len(L)
    m = len(M)
    for i in range(n):
        L[i] = L[i]*2
    for j in range(m):
        M[j] = M[j]%2

```

Java :

```java
public static void algo(int[] L, int[] M) {
    int n = L.length;
    int m = M.length;
    for (int i=0; i < n; i++){
```

```
        L[i] = L[i]*2;
    }
    for (int j=0; j < m; j++){
        M[j] = M[j]%2;
    }
}
```

(A) $O(n^2)$

(B) $O(n+m)$

(C) $O(n)$

(D) $O(2^n)$

Solution

L'algorithme est composé de 2 boucles. La première parcourt une liste `L` et multiplie par 2 les éléments de la liste. L'autre parcourt une liste `M` et assigne à chaque élément le reste de la division euclidienne de l'élément par 2. Soient n et m les tailles respectives de `L` et de `M`, on obtient une complexité de $O(n+m)$. Comme on effectue une boucle et après l'autre les complexités s'additionnent.

Question 5: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?

Python :

```python
#Entrée: L est une liste de nombre entiers
def algo(L):
    n = len(L)
    i = 0
    s = 0
    while i < math.log(n):
        s += L[i]
        i += 1
    return s
```

Java :

```java
import java.lang.Math;

public static void algo(int[] L) {
    int n = L.length;
    int s = 0;
    for (int i=0; i < Math.log(n); i++){
        s += L[i];
    }
}
```

(A) $O(n^2)$

(B) $O(n)$

(C) $O(\log(n))$

(D) $O(n^n)$

Solution

L'algorithme est composé d'une boucle qui va itérer sur $\log(n)$ éléments et va calculer la somme de ces éléments. Ainsi, l'algorithme a une complexité de $O(\log n)$. Le temps d'exécution de ce programme peut être visualisé sur la courbe jaune du graphe ci-dessous (1).

Question 6: (*10 minutes*) **Complexité** Optionnel
Quelle est la complexité du programme ci-dessous ?
Python :

```python
# Entrée: n un nombre entier
def algo(n):
    m = 0
    for i in range(n):
        for j in range(i):
            m += i+j
    return m

```

Java :

```java
public static int algo(int n) {
    int m = 0;
    for (int i=0; i < n; i++){
        for (int j=0; j < i; j++){
            m += i+j;
        }
    }
    return m;
}

```

(A) $O(n^2)$
(B) $O(n)$
(C) $O(\log(n))$
(D) $O(2^n)$

Solution

L'algorithme est composé de 2 boucles **imbriquées** suivant une suite définie par $\frac{n(n-1)}{2}$. L'algorithme va additionner les index i et j à chaque itération et les rajouter à m. Cela veut dire que nous parcourons la liste $\frac{n^2}{2} - \frac{n}{2}$, n étant la taille de la liste. Comme vu dans les conseils avant, les constantes et les termes dominés sont ignorés. La complexité de l'algorithme est ainsi de $O(n^2)$.

3 Algorithmes de Tri

Question 7: (*15 minutes*) **Insertion dans une liste triée - Python**
Soit un nombre entier `n`, et une liste triée `l`. Ecrivez un programme Python qui insère la valeur `n` dans la liste `l` tout en s'assurant que la liste `l` reste triée.

```python
def insertion_entier(liste, number):
    # TODO : Compléter ici

print(insertion_entier([2, 4, 6], 1))
```

Exemple

En passant les arguments suivants à votre programme : `n=5` et `l=[2,4,6]`. Ce dernier devra retourner `l =[2,4,5,6]`

Solution

```python
def insertion_entier(liste, number):
    # ajoute un élément à la liste
```

```
    liste.append(number)
    n = len(liste) - 1
    while n > 0 and liste[n - 1] > number:
        liste[n] = liste[n - 1]
        n -= 1
    liste[n] = number
    return liste

# solution alternative
def insertion_entier(liste,number):
    for i in range(len(liste)):
        if number <= liste[i] :
            return liste[:i] + [number] + liste[i:]
    return liste + [number]

print(insertion_entier([2, 4, 6], 1))
```

Question 8: (*30 minutes*) **Tri fusion (Merge Sort) - Python**

À partir de deux listes triées, on peut facilement construire une liste triée comportant les éléments issus de ces deux listes (leur *fusion*). Le principe de l'algorithme de tri fusion repose sur cette observation : le plus petit élément de la liste à construire est soit le plus petit élément de la première liste, soit le plus petit élément de la deuxième liste. Ainsi, on peut construire la liste élément par élément en retirant tantôt le premier élément de la première liste, tantôt le premier élément de la deuxième liste (en fait, le plus petit des deux, à supposer qu'aucune des deux listes ne soit vide, sinon la réponse est immédiate).
Les étapes à suivre pour implémenter l'algorithme sont les suivantes :

1. Si le tableau n'a qu'un élément, il est déjà trié.
2. Sinon, séparer le tableau en deux parties plus ou moins égales.
3. Trier récursivement les deux parties avec l'algorithme de tri fusion.
4. Fusionner les deux tableaux triés en un seul tableau trié.

Soit la liste `l` suivante [38, 27, 43, 3, 9, 82, 10], triez les éléments de la liste en utilisant un tri fusion. Combien d'itération effectuez-vous ?

- **Python :**

```
def merge(partie_gauche, partie_droite):
    # TODO: Code à compléter

def tri_fusion(liste):
    # TODO: Code à compléter

if __name__ == "__main__":
    l = [38, 27, 43, 3, 9, 82, 10]
    print(tri_fusion(l))
```

Conseil

- L'algorithme est récursif.
- Revenez à la visualisation de l'algorithme dans les diapositives 81 à 109 pour comprendre comment marche concrètement le tri fusion.

Conseil

Théorème 1 La récurrence

$$T(n) = a\,T(n/b) + cn^k$$
$$T(1) = c$$

où a, b, c et k sont des constantes, se résout comme suit :

$$T(n) \in O(n^k) \quad \text{si } a < b^k$$
$$T(n) \in O(n^k \log n) \quad \text{si } a = b^k$$
$$T(n) \in O(n^{\log_b a}) \quad \text{si } a > b^k$$

Solution

Python :

```python
def merge(partie_gauche, partie_droite):
    # créer la liste qui sera retournée à la fin
    liste_fusionnee = []

    # définir un compteur pour l'index de la liste de gauche
    compteur_gauche = 0
    # pareil pour la liste de droite
    compteur_droite = 0

    longueur_gauche = len(partie_gauche)
    longueur_droite = len(partie_droite)

    # continuer jusqu'à ce que l'un des index (ou les deux) atteigne l'une des longueurs (ou les deux)
    while compteur_gauche < longueur_gauche and compteur_droite < longueur_droite:
        # comparer les éléments actuels, ajouter le plus petit à la liste fusionnée
        # et augmenter le compteur de cette liste
        if partie_gauche[compteur_gauche] < partie_droite[compteur_droite]:
            liste_fusionnee.append(partie_gauche[compteur_gauche])
            compteur_gauche += 1
        else:
            liste_fusionnee.append(partie_droite[compteur_droite])
            compteur_droite += 1

    # s'il y a encore des éléments dans les listes, il faut les ajouter à la liste fusionnée
    liste_fusionnee += partie_gauche[compteur_gauche:longueur_gauche]
    liste_fusionnee += partie_droite[compteur_droite:longueur_droite]

    return liste_fusionnee  # retourner la liste fusionnée
```

Solution

```python
def tri_fusion(l):
    # compléter la fonction
    longueur = len(l)  # calculer la longueur de la liste
```

```
34      # s'il n'y a pas plus d'un élément, retourner la liste
35      if longueur == 1 or longueur == 0:
36          return l
37      # sinon, diviser la liste en deux
38      elif longueur > 1:
39          # convertir la variable en nombre entier (l'index ne peut pas ê
        tre un nombre à virgule)
40          index_milieu = int(longueur / 2)
41          # la partie gauche va du 1er élément à celui du milieu
42          partie_gauche = l[0:index_milieu]
43          # la partie droite va du milieu à la fin de la liste
44          partie_droite = l[index_milieu:longueur]
45
46          # appeler la fonction tri_fusion à nouveau sur la partie
        gauche (récursivité)
47          partie_gauche_triee = tri_fusion(partie_gauche)
48          # même chose pour la partie droite
49          partie_droite_triee = tri_fusion(partie_droite)
50
51          liste_fusionnee = merge(partie_gauche_triee,
        partie_droite_triee)  # enfin, joindre les 2 parties
52
53          # retourner le résultat
54          return liste_fusionnee
55
56
57  if __name__ == "__main__":
58      l = [38, 27, 43, 3, 9, 82, 10]
59      print(tri_fusion(l))
```

Solution

Le tri fusion est un algorithme récursif. Ainsi, nous pouvons exprimer sa complexité temporelle via une relation de récurrence : $T(n) = 2T(n/2) + O(n)$. En effet, l'algorithme comporte 3 étapes :

1. "Divide Step", qui divise les listes en deux sous-listes, et cela prend un temps constant
2. "Conquer Step", qui trie récursivement les sous-listes de taille $n/2$ chacune, et cette étape est représentée par le terme $2T(n/2)$ dans l'équation.
3. La dernière étape consiste à fusionner les listes, sa complexité est de $O(n)$.

La solution à cette équation est $O(n \log n)$.

Question 9: (*20 minutes*) **Tri à bulles (Bubble Sort) - Python** Optionnel

Le tri à bulles consiste à parcourir une liste et à comparer ses éléments. Le tri est effectué en permutant les éléments de telle sorte que les éléments les plus grands soient placés à la fin de la liste.
Concrètement, si un premier nombre x est plus grand qu'un deuxième nombre y et que l'on souhaite trier l'ensemble par ordre croissant, alors x et y sont mal placés et il faut les inverser. Si, au contraire, x est plus petit que y, alors on ne fait rien et l'on compare y à z, l'élément suivant.
Soit la liste `l = [1, 2, 4, 3, 1]`, triez les éléments de la liste en utilisant un tri à bulles. Combien d'itérations effectuez-vous ?

- **Python :**

```
1  def tri_bulle(l):
2      for i in range(len(l)):
3          #TODO: Code à compléter
4
5  if __name__ == "__main__":
6      l = [1, 2, 4, 3, 1]
7      tri_bulle(l)
```

```
8       print(l)
```

Solution

Python :

```
1  def tri_bulle(l):
2      n = len(l)
3
4      for i in range(n):
5          # Les i derniers éléments sont dans leur bonne position
6          for j in range(0, n-i-1):
7
8              # parcourir la liste de 0 à n-i-1
9              # Echanger si l'élément trouvé est supérieur
10             # au prochain élément
11             if l[j] > l[j+1] :
12                 l[j], l[j+1] = l[j+1], l[j]
13
14 if __name__ == "__main__":
15     l = [1, 2, 4, 3, 1]
16     tri_bulle(l)
17     print(l)
```

Solution

L'algorithme a une complexité de $O(n^2)$ car il contient deux boucles qui parcourent la liste.